

Azure Data Factory

Pipelines, activities, datasets, linked services, integration runtime, triggers, data movement, and production patterns

Curated study notes for the RAG learning project. Synthesized from official documentation; verify current product behavior in official docs before production use.

1. Azure Data Factory Overview

Azure Data Factory is a cloud data-integration and orchestration service for moving and transforming data. It supports ETL/ELT workflows and can dispatch work to services such as Azure Databricks and Azure SQL.

Key idea: ADF is primarily an orchestration and integration layer. It can transform data itself through mapping data flows or orchestrate external compute.

Why ADF exists

Organizations need to move data between heterogeneous sources, schedule transformations, handle dependencies, and monitor workflows. ADF provides managed pipeline authoring and execution.

- Connect many data stores.
- Coordinate activities.
- Schedule or trigger workflows.
- Monitor runs and failures.

Core components

ADF's core concepts include pipelines, activities, datasets, linked services, data flows, integration runtimes, triggers, parameters, and variables.

- Pipeline = logical workflow.
- Activity = a step in the workflow.
- Dataset = a reference to data structure/location.
- Linked service = connection information.
- Integration runtime = compute/bridge for integration work.

2. Pipelines, Activities, Datasets, and Linked Services

These objects form the basic mental model for ADF. Understanding their relationships makes pipeline design much easier.

Key idea: A linked service answers 'how do I connect?' while a dataset answers 'what data/location does this activity use?'

Pipeline

A pipeline groups activities into a logical unit of work. Activities can be sequential, conditional, or parallel depending on dependencies.

- Keep pipelines understandable.
- Use child pipelines for reusable workflows where appropriate.
- Parameterize reusable logic.

Activities

Activities perform actions. Copy activity moves data; transformation activities transform it; control activities implement logic such as loops, conditions, waits, and execution of other pipelines.

- Activities can have success/failure/completion dependencies.
- Retries should be used deliberately.
- Use secure inputs/outputs for sensitive values where supported.

Datasets and linked services

Datasets represent input/output data structures; linked services define connection information to stores or compute.

- One linked service can support multiple datasets.
- Parameterize datasets for reusable paths/tables.
- Do not hard-code credentials in activity code.

3. Integration Runtime, Triggers, Parameters, and Monitoring

The Integration Runtime provides the compute infrastructure/bridge used by ADF for data movement, data flow execution, and dispatching transformation activities.

Key idea: IR choice affects connectivity, network placement, performance, and security.

Integration Runtime

ADF provides managed Azure IR for cloud integration and self-hosted IR for scenarios requiring access to private/on-premises resources. Exact capabilities vary by workload.

- Choose the IR based on network and execution requirements.
- Place self-hosted IR close to private data where appropriate.
- Monitor IR health and throughput.

Triggers

Triggers determine when a pipeline run begins. Common patterns include scheduled, tumbling-window, event-based, and manual execution.

- Use event triggers when data arrival is the dependency.
- Use tumbling windows when time-window processing needs stateful scheduling.

Parameters and variables

Parameters provide run-time configuration; variables store temporary state within pipelines.

- Use parameters for reusable pipeline design.
- Keep secrets in secure mechanisms rather than parameters when possible.
- Log safe parameter values for debugging.

Monitoring

ADF monitoring shows pipeline runs, activity runs, duration, errors, and trigger history.

- Capture failure messages.
- Use alerts for important failures.
- Track SLA/freshness rather than only task success.

4. Copy Activity, Data Flows, and ADF Design Patterns

ADF supports data movement through Copy Activity and transformations through Mapping Data Flows or external compute such as Databricks.

Key idea: Use ADF to orchestrate the right compute. Do not force every transformation into the same tool.

Copy Activity

Copy Activity moves data between supported stores and can handle mapping, format conversion, and transfer settings.

- Use source/sink datasets or parameterized connections.
- Validate row counts and data types for critical transfers.
- Consider incremental copy patterns for large sources.

Mapping Data Flows

Mapping Data Flows provide visual transformation graphs that execute on managed Spark infrastructure.

- Useful for teams that prefer visual authoring.
- Understand that transformations still have compute and cost.
- Use Databricks when advanced Spark engineering or shared lakehouse logic is more appropriate.

External compute

ADF can orchestrate Azure Databricks, Synapse, SQL, and other compute services.

- ADF coordinates; specialized services compute.
- Pass parameters between orchestration and transformation layers.
- Design clear ownership of retries.

5. Production ADF Checklist

A production ADF solution needs more than a working copy pipeline. It should be parameterized, observable, secure, testable, and designed for reruns.

Key idea: Treat ADF pipelines as software: version them, test them, monitor them, and document their failure/recovery behavior.

Reliability

Make activities idempotent where possible. Define retry policy and failure paths. Design backfills and reruns explicitly.

- Avoid duplicate appends on retry.
- Use deterministic partitions or upserts.
- Quarantine bad input where appropriate.

Security

Use managed identity/service principals and Key Vault integration where supported. Restrict network access and follow least privilege.

- Separate development and production identities.
- Do not expose secrets in logs.
- Use private connectivity where required.

Operations

Monitor pipeline duration, failures, data volumes, and freshness. Use alerts for meaningful incidents.

- Document owners and dependencies.
- Record source/target and schedule.
- Test failure recovery.

Reference pattern

A common Azure architecture is ADF → ADLS/SQL sources → Databricks/Synapse transformation → curated storage/warehouse → BI.

- Use ADF for orchestration.
- Use Databricks for Spark-heavy transformations.
- Use SQL/Synapse for serving and analytics where appropriate.

Selected official sources

- <https://learn.microsoft.com/en-us/azure/data-factory/introduction>
- <https://learn.microsoft.com/en-us/azure/data-factory/concepts-integration-runtime>
- <https://learn.microsoft.com/en-us/azure/data-factory/concepts-datasets-linked-services>
- <https://learn.microsoft.com/en-us/azure/data-factory/>